Lecture 3: OS Overview

Professor G. Sandoval

Some slides adapted by G. Sandoval for CS6233, from Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.
Also from Slides by Brendan Dolan-Gavitt

XV6

User/Kernel/System Calls

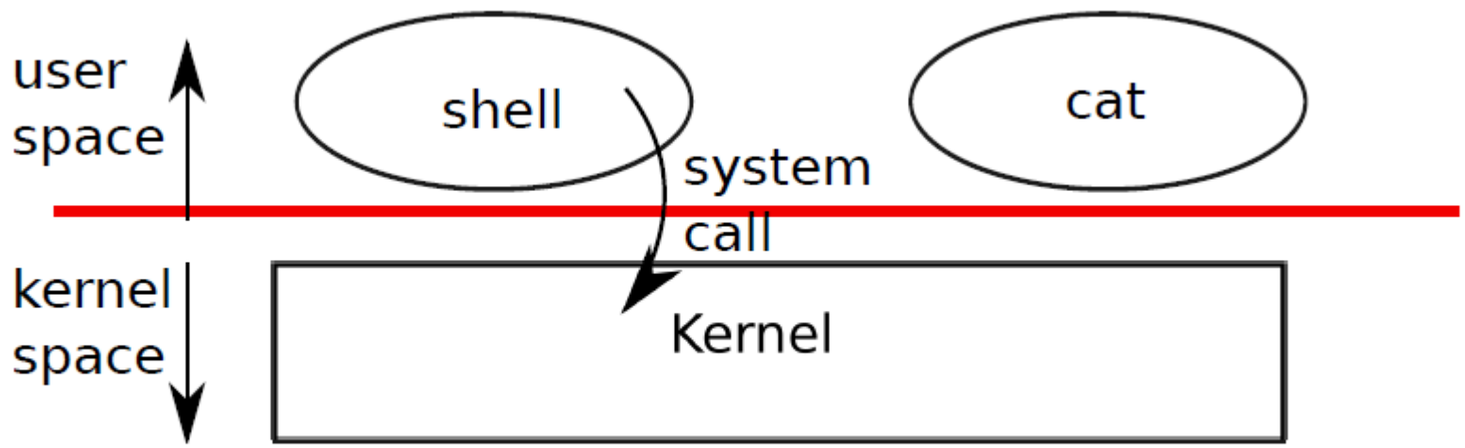


Figure 0-1. A kernel and two user processes.

User & Kernel

- The ***kernel*** is a program at the core of the **operating system.**

- ***User* space** is where ordinary programs run; must trap to the kernel (via a *system call*) to do anything that requires privilege

- User space communicates with the kernel via **system calls.**

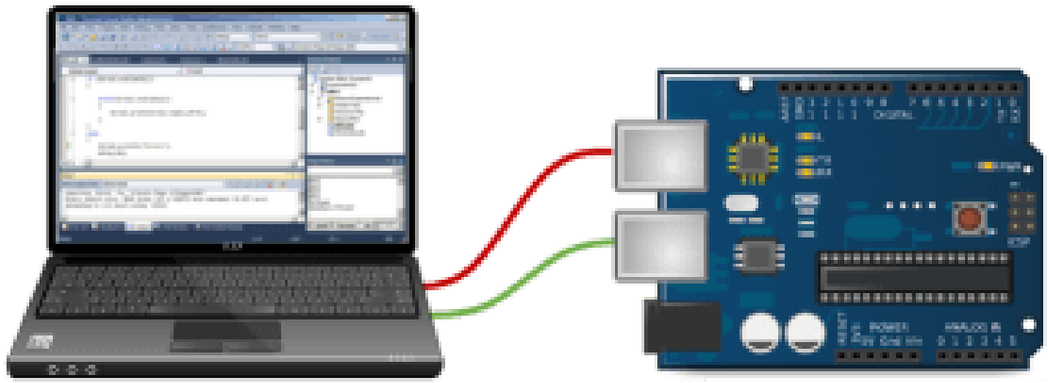
System call	Description
fork()	Create process
exit()	Terminate current process
wait()	Wait for a child process to exit
kill(pid)	Terminate process pid
getpid()	Return current process's id
sleep(n)	Sleep for n seconds
exec(filename, *argv)	Load a file and execute it
sbrk(n)	Grow process's memory by n bytes
open(filename, flags)	Open a file; flags indicate read/write
read(fd, buf, n)	Read n byes from an open file into buf
write(fd, buf, n)	Write n bytes to an open file
close(fd)	Release open file fd
dup(fd)	Duplicate fd
pipe(p)	Create a pipe and return fd's in p
chdir(dirname)	Change the current directory
mkdir(dirname)	Create a new directory
mknod(name, major, minor)	Create a device file
fstat(fd)	Return info about an open file
link(f1, f2)	Create another name (f2) for the file f1
unlink(filename)	Remove a file

XV6

- XV6 is a **UNIX like** operating system developed at MIT
- Reimplements **UNIX version 6** (1975)
- Small enough to cover in a semester
- **Few system calls**
- Lacks many features (networking, users, security features, etc.)

Simulating a PC

- Real operating systems are difficult to debug
- If something goes wrong, the whole machine crashes



- At a minimum you need **two computers**

Simulating a PC

- To get around this, we will use a PC emulator called QEMU
- QEMU simulated PC hardware in software
- We will call the machine doing the simulating the “host” and the machine being simulated the “guest”

XV6 Notes

Unix/XV6 Notes

- We will briefly cover some XV6 aspects:
  - Command Line arguments
  - File I/O
  - Pipes and redirection

Command Line Arguments

- When we launch an executable the OS sets up it's environment to run
- For traditional UNIX style C Programs this includes a list of command arguments
- Command line args are passed to the main function
- `int main(intargc, char *argv[])`

Command Line Arguments

- Example: suppose you execute

  `$ wc README`

- argv[0] = ?
- argv[1] = ?
- argv[2] = ?
- argc = ?

Command Line Arguments

- Example: suppose you execute

  `$ wc README`

- argv[0] = "wc"
- argv[1] = "README"
- argv[2] = **NOTHING**
- argc = 2

Include Files

- At a Minimum you will need
  - **Types.h** – basic data types used in XV6
  - **User.h** – defines the API accessible by programs running in user mode
- If you want to find out what to include to get a particular function: **`grep function *.h`**

I/O and File Descriptors

- A File Descriptor is a small integer representing a “kernel managed” object that a process can read/write to.
- A process can obtain a FD by: opening file, directory, or device or creating a pipe.
- The FD interface abstracts away differences between files, pipes and devices making them all look like a stream of bytes.

File Descriptors

- The shell always ensures that any process has 3 file descriptors open:
- The 3 standard file descriptors:
  - stdin(0),
  - stdout(1),
  - stderr(2)

Open

- In XV6 you have the open syscall:
- The mode argument is a set of flags (defined in fcntl.h): O_RDONLY, O_WRONLY, O_RDWR, O_CREATE
- Returns a file descriptor, or -1 if there is an error (e.g., file not found)

UNIX I/O: read and write

- Once you have open file descriptor, it can be used with various other APIs:
  - int printf(int fd, char *format, ...)
  - int read(int fd, void *buf, int n)
  - int write(int fd, void *buf, int n)

UNIX I/O: read and write

Data to read or write

- Once you have open file descriptor, it can be used with various other APIs:
  - int printf(int fd, char *format, ...)
  - int read(int fd, void *buf, int n)
  - int write(int fd, void *buf, int n)

UNIX I/O: read and write

Number of bytes to read or write

- Once you have open file descriptor, it can be used with various other APIs:
  - int printf(int fd, char *format, ...)
  - int read(int fd, void *buf, int n)
  - int write(int fd, void *buf, int n)

Redirection

- At the command line, a program's std input and output can be redirected to or from a file
  - Input with the '<' character
  - Output with the '>' character

- Example: search for the word 'the' in the file README
  - Grep the README > found.txt
- Now save the results to a file "found.txt"
  - Grep the < README > found.txt

Takes place of stdin Takes place of stdout

Pipes

- Since input and output use the general notion of a file descriptor, we can do interesting things.
- This way of connecting programs is called a *pipe* –it hooks up the output of one program to the input of another

Pipes

- Examples:
  - Count the lines, words, and characters in a file:
    - `cat README | wc`
  - Search the dictionary for words containing "cat" and return only the first 10
    - `grep cat /usr/share/dict/words | head`

cat utility example

- cat doesn't know where it's reading from
- Use of file descriptors and the convention that fd 0 is input and fd 1 is output allows for a simple implementation of cat.
- When using redirection the shell takes care of this

```
char buf[512];
int n;

for(;;){
  n = read(0, buf, sizeof buf);
  if(n == 0)
    break;
  if(n < 0){
    fprintf(2, "read error\n");
    exit();
  }
  if(write(1, buf, n) != n){
    fprintf(2, "write error\n");
    exit();
  }
}
```

Basic OS Concepts

- User and kernel space
- Processes
- System Calls
- Address spaces
- Files & file descriptors
- Users, groups, permissions (UNIX)

User/Kernel/System Calls

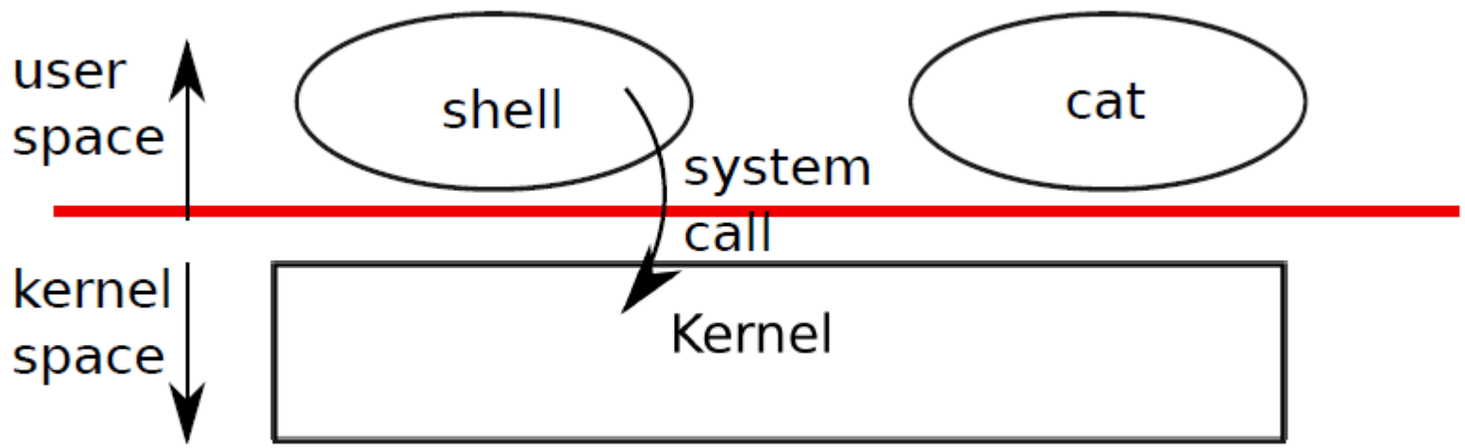


Figure 0-1. A kernel and two user processes.

User & Kernel

- The ***kernel*** is where we'll spend most of our attention

- ***User* space** is where ordinary programs run; must trap to the kernel (via a *system call*) to do anything that requires privilege

When poll is active, respond at **pollev.com/gsandoval**
Text **GSANDOVAL** to **22333** once to join

User Space and Kernel Space Communicate via?

- Pipes
- Redirection
- System calls
- Command Line Arguments
- None of the above

Powered by Poll Everywhere
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

What Goes in the Kernel?

- distinguishes several types of operating systems
- Some types proposed over the years:
  - Monolithic
  - Microkernel
  - Virtual machine

Monolithic Kernels

- ***Monolithic kernels*** are effectively one large program; any piece of the kernel can talk to any other by simple function calls

- This is the **most common** kind in use today – Windows, Linux, and OS X are all monolithic

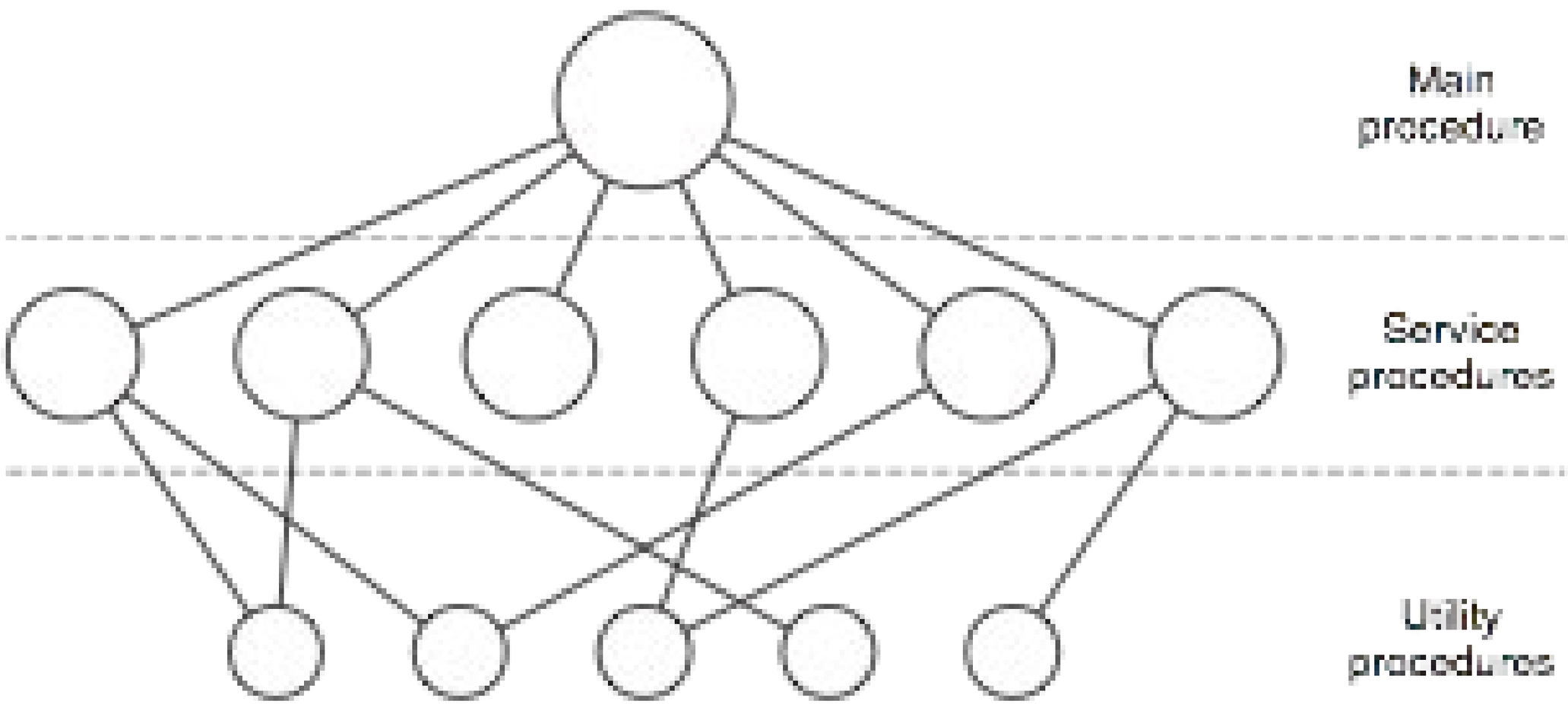
- **Downside**: any mistake in the kernel can bring down the entire machine

:(

Your PC ran into a problem and needs to restart. We're just collecting some error info, and then we'll restart for you. (0% complete)

If you'd like to know more, you can search online later for this error: HAL_INITIALIZATION_FAILED

Monolithic Organization



Microkernels

- **Core idea:** bugs per lines of code seems to be roughly constant

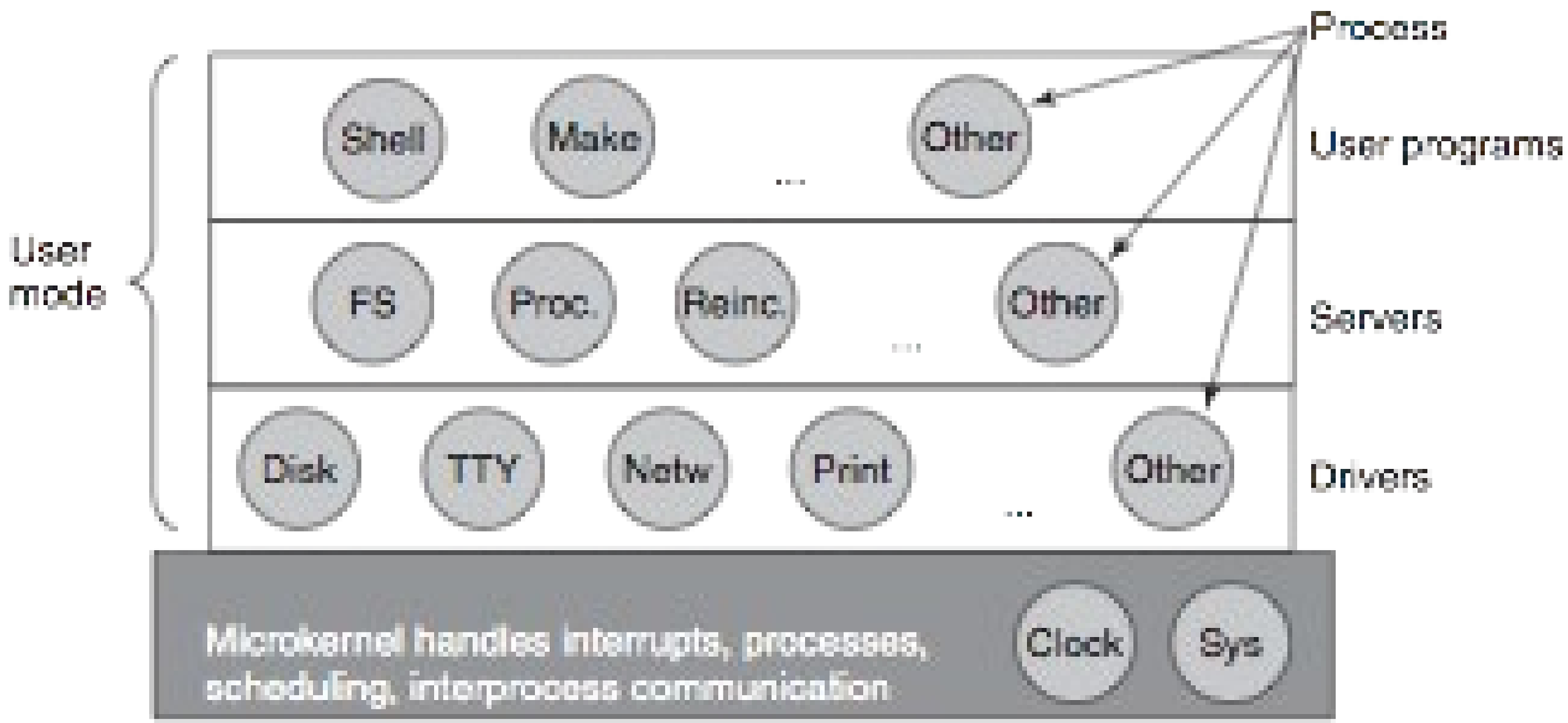
- fewer bugs? write less code!

- **Corollary**: make the kernel as small as possible

Microkernels

- OS is structured as a **collection of servers** running in user space
- Kernel-mode component just **does scheduling** and inter-process communication
- **Stability benefits:** your video driver can crash without bringing down the whole OS

Microkernel Organization



Microkernel Downsides

- **Communication** between processes requires a ***context switch*** – and this is expensive on current hardware
- Lots of work has been done to make this. But it still *appears* to be inherently **slower** than a monolithic kernel

Aside: Microkernels on Your PC

- Some modern OSes *do* have microkernel-like aspects
- **Windows**
  - Some subsystems (e.g. the local security manager, LSASS) are **regular user processes**
  - In early versions of Windows NT (up until 4.0), the graphics subsystem was entirely in **userspace**
  - But this was **later moved into the kernel** (win32k.sys) for performance reasons

Process

- **Definition**: A program in execution.
- A process is the **basic unit of execution in an** Operating System
- Each process gets its own **CPU context** and **memory**
- The operating system **switches** between them, **saving** and **restoring** their context

Address Spaces

- Each process gets its **own memory** and cannot access the memory of others
  - Except through special mechanisms such as shared memory regions
  - This is **enforced** by the Operating System

Address Spaces

- Two ways of accomplishing **Isolation** of Address Spaces:
  - **Naming**: make sure that a process has no way of referencing the memory of another process; i.e., it can't form a memory address that refers to memory outside its own space
  - **Trapping**: ensure that every access to someone else's memory causes a trap to the kernel

Files

- One of the **most fundamental units** in most operating systems
- Usually just a **plain array of bytes** with some associated metadata
- Programs can obtain a reference to a file from the operating system (called a ***file descriptor*** or ***file handle***)
- All access to the file data is done by asking the OS and providing it the descriptor

Files in UNIX

- UNIX takes the mentality that **"everything's a file"**
- Some hardware resources can be accessed by user programs by operating on files instead
- For example: in Linux, /dev/ttyS0 is a file that gives you access to the serial port, **/dev/hda** is a file that refers to the actual **hard drive**

Files in UNIX

- Even program input and output is treated as a file:
  - ***Standard input*** is assigned to file descriptor 0
  - ***Standard output*** is assigned to file descriptor 1
  - ***Standard error*** is assigned to file descriptor 2

Users and Groups

- We may want to allow multiple people to use the same system
- So the OS needs to track the **owners** of resources such as files
- It may also be convenient to have files accessible to a ***group*** of users

UNIX File Permissions

- Each file has an *owner*, a *group*, and *permissions*
- Permissions divided into three sections: *user*, *group*, and *others*

Permissions **Owner** **Group**

```
-rw-r--r--     1 moyix  staff     8388608 Sep  1 18:00 vga.vram
-rwxr-xr-x     1 moyix  staff        8496 Sep 14 07:34 x
-rw-r--r--     1 moyix  staff         290 Sep 14 07:34 x.c
```

Permissions

-rwxr-xr-x

User Group Other

- If the file is **owned** by the **current user,** use the ***user* permissions** to determine if access is allowed
- Otherwise, if the user is a **member** of the **group** of the file, use the ***group* permissions**
- Finally, **anyone else** will be allowed access according to the ***other*** permissions
- In this example, the file's owner can read, write, and execute the file; the group and others can only read and execute it but not modify it
- When you see a – (dash) in user, group or owner it means the particular permission has NOT been granted.

Permissions

chmod

- Use the chmod command to change file permissions.
- Take a look at the file first. At the shell prompt type:
- ls –l foo.txt

```
ls -l foo.txt
```

- This displays:

```
-rw-rw-r-- 1 user user 150 Mar 19 08:08 foo.txt
```

- Now type:

```
chmod o+w foo.txt
```

- Result:

```
-rw-rw-rw- 1 user user 150 Mar 19 08:08 foo.txt
```

Permissions

Identities

u — the user who owns the file (that is, the owner)
g — the group to which the user belongs
o — others (not the owner or the owner's group)
a — everyone or all (u, g, and o)

Permissions

r — read access
w — write access
x — execute access

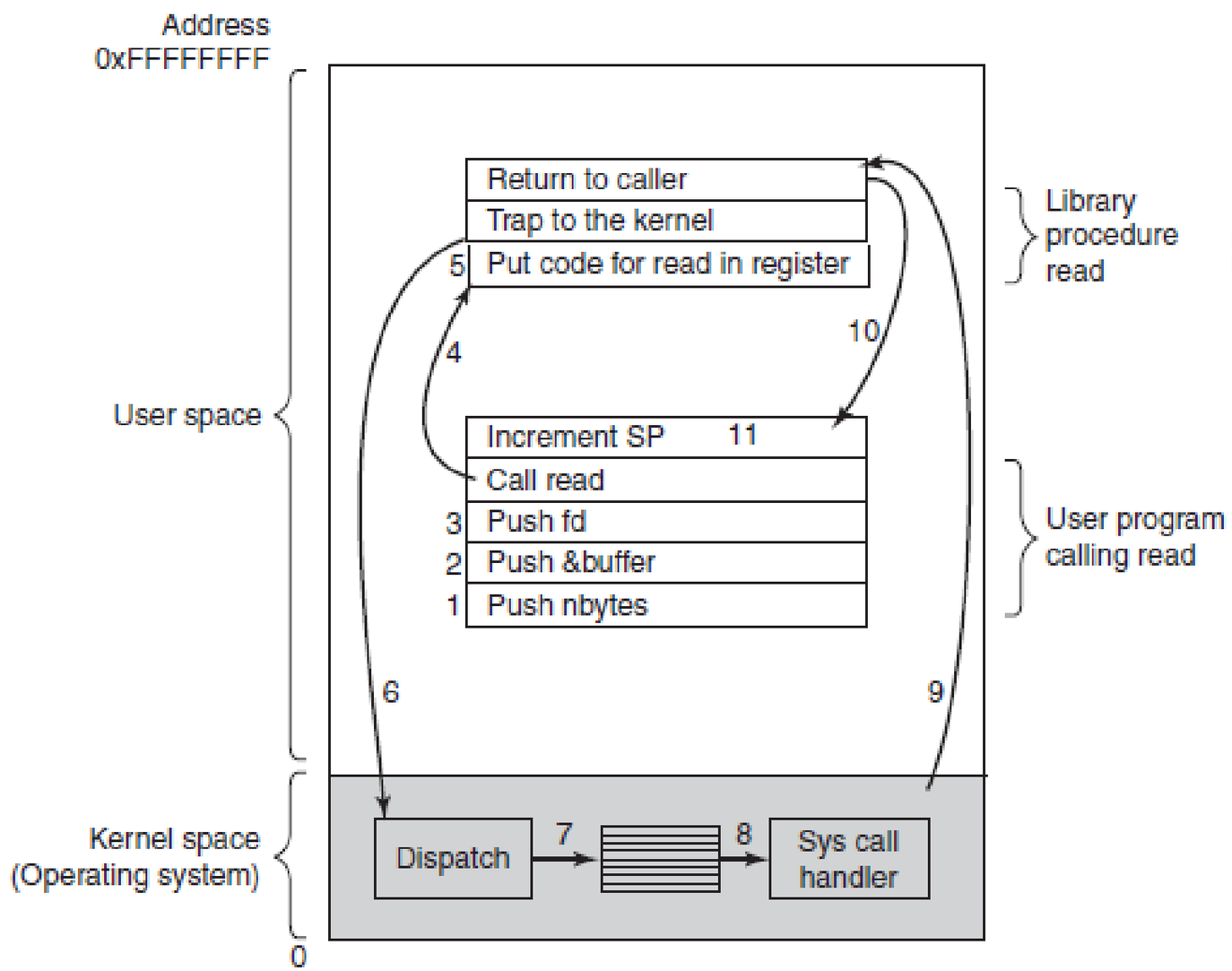
Actions

+ — adds the permission
- — removes the permission
= — makes it the only permission

System calls

- Abstractions that OS provides to interact with it.
- Are usually machine dependent
- Written in assembly code
- Library procedures are used in C/C++.
- Let’s trace the read system call:
  - `Count = read(fd, buffer, nbytes)`

System Calls



The 11 steps in making the system call *read(fd, buffer, nbytes).*